

Alternatīvs veids eksāmena kārtīšanai studiju kursā Objektorientētā programmēšana

Tiek piedāvāta alternatīva iespēja kārtot eksāmenu, aizstāvēt patstāvīgi izveidotu programmu. Šī iespēja ir piemērota studentiem, kuriem jau ir laba pieredze programmēšanā un programmu veidošanā Microsoft Visual Studio vidē valodā C++.

Patstāvīgi izstrādātā programma un tās aizstāvēšana tiks novērtēta ar atzīmi, kas tiks izmantota kā eksāmena atzīme. Studiju kursa gala atzīme tiks aprēķināta pēc prasībās noteiktās formulas.

Uzdevums

Jāizveido futbola komandas simulācijas programmatūra. Studenta uzdevums ir patstāvīgi izveidot nepieciešamās klases un nodrošināt šo klašu objektu interfeisu atbilstoši zemāk dotajam aprakstam. Uzdevums jāizpilda Microsoft Visual Studio vidē (versija ne zemāka par Microsoft Visual Studio Community 2022) valodā C++, izmantojot tikai standarta programmēšanas valodas C++ līdzekļus un MFC bibliotēkas iespējas. Studenta izveidotajam izpildāmajam kodam jābūt dinamisko saišu bibliotēkas (DLL) formātā, kas izmantojama ar doto spēles simulācijas programmu GameSimulator.exe.

Pieteikšanās un izpildes kārtība

1. Ja students pieņem lēmumu pieteikties alternatīvajam eksāmenam, tad **līdz 20.04.2026. plkst. 13.00** jāatsūta pieteikuma vēstule uz e-pasta adresi uldis.sukovskis@rtu.lv, norādot vēstulē vārdu, uzvārdu, studenta apliecības numuru. Vēstulē jāuzraksta apliecinājums, ka students piekrīt šā uzdevuma izpildes nosacījumiem, ka apņemas uzdevumu veikt patstāvīgi, neizmantojot citu personu vai mākslīgā intelekta rīku radītu programmas pirmkodu, kā arī apņemas iesniegt gatavo programmatūru un tās pirmkodu atbilstoši iesniegšanas prasībām un norādītajos termiņos.
2. Uzdevums jāizpilda un visi norādītie faili jāiesniedz, augšupielādējot tos e-studiju kursā paredzētajā iesniegšanas vietā **līdz 12.05.2026 plkst. 23.59**. Jāiesniedz pilns Visual Studio projekts, kas arhivēts ZIP formātā (ZIP faila lielums nedrīkst pārsniegt 50 MB; lai samazinātu ZIP faila apjomu, izdzēsiet no projekta mapes apakšmapēm liela izmēra datubāzes u. tml. failus), kā arī jāiesniedz ZIP formātā arhivēti atsevišķi TeamLeft.dll un TeamRight.dll faili, kas izveidoti šajā Visual Studio projektā kā *Release* versija. Ja līdz noteiktajam termiņam darbs netiek iesniegts, tad students var kārtot studiju kursa eksāmenu grafikā noteiktajā datumā, un tā tiks uzskatīta par pirmo eksāmena kārtīšanas reizi.
3. Programmas pirmkodā obligāti jābūt komentāriem, kas paskaidro programmas mainīgo nozīmi un algoritmu darbību. Klašu, atribūtu un metožu nosaukumi jāveido tā, lai tie būtu viegli uztverami un raksturotu to lietojumu programmā.
4. Ja iesniegtais izpildāmais kods (t. i., TeamLeft.dll un TeamRight.dll) darbojas atbilstoši aprakstā noteiktajai funkcionalitātei, iesniegto Visual Studio projektu ir iespējams izmantot izpildāmā koda veidošanai standarta Visual Studio vidē ar MFC bibliotēku, programmas pirmkods ir izveidots un noformēts atbilstoši prasībām, tad studentam tiks nozīmēts aizstāvēšanas datums un laiks, nosūtot uzaicinājuma vēstuli uz studenta e-pasta adresi ne vēlāk kā divas dienas pirms darba aizstāvēšanas. Ja iesniegtais darbs neatbilst prasībām, tad tiek reģistrēta nesekmīga eksāmena atzīme, un students varēs kārtot studiju kursa atkārtoto eksāmenu atbilstoši RTU noteiktajai kārtībai.
5. Iesniegtās programmas aizstāvēšanas notiek klātienē. Tās laikā students uz sava personālā datora demonstrē un skaidro savas programmas darbību, atbild uz jautājumiem par pirmkodu, veic izmaiņas pirmkodā un skaidro objektorientētās programmēšanas konceptu izmantošanu. Aizstāvēšana un iesniegtā darba kvalitāte tiek novērtēta ar atzīmi 10 ballu skalā. Atzīme tiek izmantota kā eksāmena atzīme studiju kursa gala atzīmes aprēķināšanas formulā. Ja darba aizstāvēšanā saņemta atzīme ir nesekmīga, tad tāpat kā parastā eksāmena gadījumā šī atzīme ir studiju kursa gala atzīme, un students varēs kārtot studiju kursa atkārtoto eksāmenu atbilstoši RTU noteiktajai kārtībai.

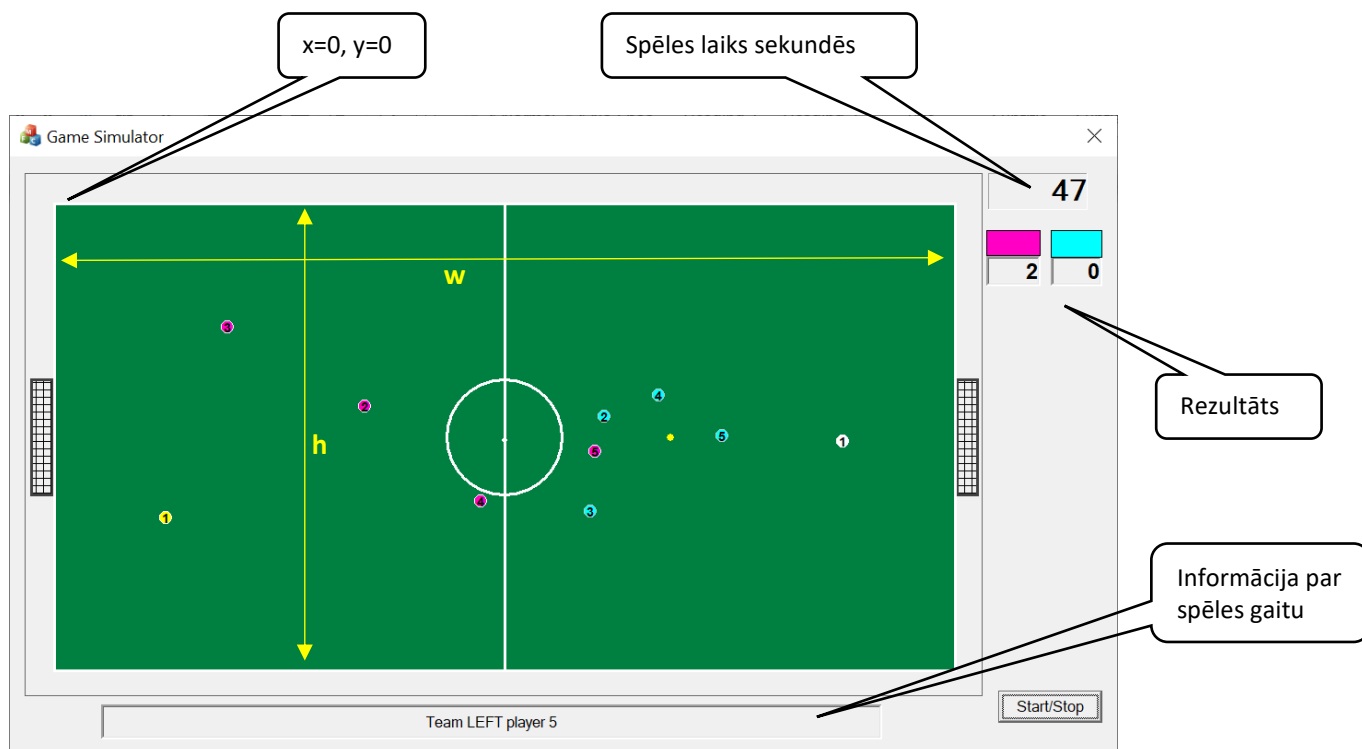
Gatavās komponentes uzdevuma izpildei

1. Uzdevuma izpildei var izmantot doto MS Visual Studio projekta sagatavi, kas pieejama failā **GameSimulation_0.zip**. Tajā ir Visual Studio projekts TeamLeft, kas konfigurēts DLL izveidošanai un kurā ir obligāto interfeisu deklarācijas (failos GameInterface.h un PlayerInterface.h), kā arī iesākti paraugi dažu komandas spēlētāju klašu realizācijai (Player.h, Player.cpp, Goalkeeper.h un Goalkeeper.cpp).
2. Sava koda darbības pārbaudei jāizmanto dotā spēles simulācijas vadības programma GameSimulator.exe. Komandu realizācijas bibliotēkām TeamLeft.dll un TeamRight.dll pēc noklusējuma jābūt tajā pašā failu mapē, kur GameSimulator.exe. Failos **GameTestBed86.zip** un **GameTestBed64.zip** (attiecīgi x86 un x64 konfigurācijām) ir spēles simulācijas vadības programma GameSimulator.exe, kā arī gatavas TeamLeft.dll un TeamRight.dll *Release* versijas, kuras var izmantot kā pretinieka komandas savas spēlētāju komandas pārbaudei (šajos DLL paraugos ir īstenota visai vienkārša spēlētāju darbība laukumā).

Spēles principu īss apraksts

1. Spēle notiek laukumā, kas līdzīgs futbola laukumam.
2. Vienas spēles ilgums ir 60 sekundes. Komandas nemainās ar laukuma pusēm.
3. Spēlē piedalās divas komandas, katrā 5 spēlētāji. Komandā ir ne mazāk kā trīs veidu spēlētāji ar atšķirīgu uzvedību – viens vārtsargs, viens vai vairāki aizsardzības spēlētāji, viens vai vairāki uzbrucēji.
4. Spēles sākumā spēles simulācijas vadības programma izvieto katras komandas spēlētājus nejauši ģenerētās pozīcijās vienā no laukuma pusēm un novieto bumbu laukuma centrā.
5. Pārvietojoties pa laukumu, spēlētāji cenšas nesadurties. Izņēmums var būt vārtsargs, kurš drīkst sadurties ar laukuma spēlētāju.
6. Ja bumba ir tuvu spēlētājam (attālums no spēlētāja centra līdz bumbas centram ir ne lielāks par spēlētāja diametru), tad spēlētājs var izdarīt sitienu pa bumbu.
7. Bumba pārvietojas tikai laukuma plaknē, un tās kustības ātrums pēc sitiena pakāpeniski samazinās.
8. Bumba nešķērso laukuma malas, bet atsitas no tām līdzīgi kā no hokeja laukuma bortiem. Ja bumba trāpa vārtos un tās centrs šķērso vārtu līniju, tad tiek ieskaitīti vārti.
9. Spēles atsākšanai pēc vārtu gūšanas spēles simulācijas vadības programma komandu spēlētājus izvieto savā laukuma pusē nejauši ģenerētās pozīcijās, bet viens laukuma spēlētājs no vārtus guvušās komandas tiek novietots sitiena attālumā no bumbas, kas tiek novietota laukuma centrā.
10. Spēli sāk, uz laiku var apturēt un atkal atsākt, izmantojot pogu "Start", "Pause" un "Resume".
11. Kreisās un labās laukuma puses komandu spēlētāju objektu izveidošanu un uzvedību spēles laikā nodrošina izpildāmais kods no dinamisko saišu bibliotēkām, kuru failu nosaukumiem jābūt TeamLeft.dll un TeamRight.dll. Spēles simulācijas vadības programma GameSimulator.exe šīs bibliotēkas izmanto, lai izveidotu spēlētāju objektus kreisās puses un labās puses komandai un lai spēles gaitā sazinātos ar spēlētāju objektiem, izmantojot klasē PlayerInterface deklarētās metodes.

Spēles laukums



Spēles parametru vērtības

1. Spēlētājs tiek attēlots ar aplīti, kura diametrs ir no 10 līdz 20 ekrāna vienībām (pikseļiem). Spēlētāja koordinātes ir tā centra koordinātes. Katram spēlētājam ir jābūt numuram (vesels skaitlis), kas jāattēlo aplīša iekšpusē. Spēlētāja attēla zīmēšana jānodrošina spēlētāja klases metodē `Draw()`. Spēlētāja krāsu nosaka spēles vadības simulācijas programma, kad pirms spēles izveido spēlētāja objektu. Tikai vārtsargam atļauts izmantot citu krāsu sava attēla zīmēšanai. Krāsa tiek apzīmēta ar veselu skaitli, kura vērtība tiek veidota ar makrodefinīciju `RGB(red,green,blue)`, kur *red*, *green* un *blue* ir vesels skaitlis no 0 līdz 255, kas nosaka sarkanās, zaļās un zilās krāsas komponentes intensitāti.
2. Vārtsargam jāpiešķir numurs 1, pārējiem spēlētājiem numura vērtība var būt jebkura, bet tai jābūt saredzamai spēlētāja attēla aplīti.
3. Bumba tiek attēlota ar aplīti, kura diametrs ir 8 ekrāna vienības (pikseļi). Bumbas koordinātes ir tās centra koordinātes. Bumbas attēlošanu un pārvietošanos nodrošina spēles simulācijas vadības programma.
4. Spēles laukuma augšējā kreisā stūra koordinātes ir (0,0).
5. Spēles laukuma taisnstūra platumu un augstumu spēlētājs var iegūt, izmantojot klasē `GameInterface` deklarēto metodi `GetFieldSize(int& w, int& h)`. Vārtu platumu var uzzināt, izmantojot metodi `GetGateWidth()`.
6. Spēles simulācijas programma `GameSimulator.exe` maina visu laukumā esošo spēlētāju un bumbas novietojumu (koordinātes) ik pēc vienas spēles laika vienības (50 milisekundēm), proti, spēles laikā ik pēc 50 ms tiek izsaukta katra spēlētāja metode `Move()`. Šajā metodē katram spēlētājam jāaprēķina savas nākamās pozīcijas koordinātes. Spēles simulācijas programma `GameSimulator.exe` uzzinās katra spēlētāja atrašanās vietu, izmantojot spēlētāja objekta metodi `GetPosition()`.
7. Spēlētāju un bumbas ātrums tiek mērīts ekrāna vienībās (pikseļos) vienā spēles laika vienībā (50 ms).
8. Spēlētāja ātrums nedrīkst pārsniegt 30 pikseļus vienā spēles laika vienībā.
9. Sitot pa bumbu, spēlētājs tai piešķir sākuma ātrumu un leņķi, kuru mēra grādos. Sitiens no laukuma kreisās puses uz labo pusi paralēli laukuma sānu malām ir 0 grādu leņķi. Sitiens no laukuma apakšējās sānu malas virzienā uz augšējo paralēli gala līnijai ir 90 grādu leņķi, bet no augšējās uz apakšējo ir 270 grādu leņķi. Sitiens pa bumbu nedrīkst tai piešķirt ātrumu, kas pārsniedz 50. Pēc sitiena pa bumbu tās ātrums pakāpeniski samazinās.
10. Spēles simulācijas vadības programma `GameSimulator.exe` spēles gaitā ieraksta failu `GameLogFile.csv`, kurā fiksē ātruma un citus pārkāpumus. Nelieli pārkāpumi ir pieļaujami (dažkārt pārsniegts ātrums, dažas spēlētāju saskriešanās), un par tiem pienākas tikai brīdinājums, nevis noraidījums :).
11. Spēle ilgums ir 60 sekundes.

Interfeisu apraksts

1. Spēlētājs var iegūt informāciju par spēles gaitu un parametriem, izmantojot klasē `GameInterface` deklarētās metodes.
2. Spēles simulācijas vadības programma `GameSimulator.exe` sazinās ar spēlētājiem, izmantojot klasē `PlayerInterface` deklarētās metodes, kuru realizācijai jābūt spēlētāju darbību īstenojošajās klasēs.
3. Spēlētāja klasē jābūt realizētām visām metodēm, kas ir deklarētas kā tīrās virtuālās metodes klasē `PlayerInterface`. Spēlētāja klasei jābūt atvasinātai no klases `PlayerInterface`. Var veidot apakšklases dažāda veida spēlētājiem (vārtsargam, uzbrucējiem, aizsardzības spēlētājiem).
4. Spēles simulācijas vadības programma `GameSimulator.exe` pirms spēles sākuma katra spēlētāja instances (objekta) radīšanai izsauc funkciju `CreatePlayer()`, kurai nodod trīs parametrus – rādītāju uz klases `GameInterface` instanci (šo rādītāju spēlētājs var izmantot spēles gaitā piekļuvei klases `GameInterface` metodēm, lai iegūtu nepieciešamo informāciju savas rīcības noteikšanai), spēlētāja komandas laukuma pusi (kreisā vai labā) un spēlētāja krāsu. Šai funkcijai jābūt realizētai komandu dinamisko saišu bibliotēkās (DLL) `TeamLeft.dll` un `TeamRight.dll`. Funkcijas `CreatePlayer()` paraugs ir iekļauts projekta sagatavē. Funkcija jāveido atbilstoši šim paraugam (paraugā pieņemts, ka spēlētāju klases vārds ir `Player` un ka vārtsarga klases vārds ir `Goalkeeper`, bet šīm klasēm, protams, var būt citādi nosaukumi). Spēlētāja numura veidošanu, kā arī citu atribūtu vērtību piešķiršanu un apstrādi var veidot atšķirīgi no paraugā dotajiem piemēriem.
5. Pirms spēles sākuma un pēc gūtiem vārtiem spēles simulācijas vadības programma, izpildot spēlētāja interfeisa metodi `SetPosition()`, novieto katru spēlētāju attiecīgās komandas laukuma pusē vadības programmas aprēķinātās koordinātēs.
6. Spēles gaitā ik pēc vienas spēles laika vienības (50 ms) spēles simulācijas vadības programma katram spēlētājam izpilda tā metodi `Move()`, kurā spēlētājs aprēķina savas nākamās pozīcijas koordinātes.
7. Spēlētāja klases realizācijas piemēra uzmetums:

Player.h

```
class Player : public PlayerInterface {
protected:
    GameInterface* pGame;
    int myID;
    COLORREF myColor;
    //...
public:
    //...
}
```

Player.cpp

```
static int IDNUM = 0; // statisks mainīgais spēlētāju numuru veidošanai
```

```
// Funkcija spēlētāju objektu radīšanai, kuru pirms spēles sākuma izsauc spēles vadības programma GameSimulator.exe
// Pirms spēles sākuma spēles vadības programma šo funkciju izsauc 5 reizes katrai no komandām, lai radītu spēlētājus.
// Šī funkcija atgriež rādītāju uz izveidoto spēlētāja objektu kā bāzes klases PlayerInterface rādītāju.
```

```
// Funkcijas nosaukumam, atgriežamās vērtības tipam un parametriem precīzi jāatbilst šim piemēram.
```

```
extern "C" __declspec(dllexport) PlayerInterface* _cdecl CreatePlayer(GameInterface* pG, Side side, COLORREF color)
{
    ++IDNUM;
    if (IDNUM == 1) return new Goalkeeper(pG, side, color);
    //...
    return new Player(pG, side, color);
}
```

```
Player::Player(GameInterface* pG, Side side, COLORREF color) { // Klases Player konstruktors
    pGame = pG; // Piešķir spēlētāja atribūtam pGame rādītāju piekļuvei klases GameInterface metodēm
    myColor = color; // Piešķir spēlētāja atribūtam myColor no vadības programmas saņemto krāsas vērtību
    mySide = side; // Piešķir spēlētāja atribūtam mySide no vadības programmas saņemto laukuma puses vērtību
    myID = IDNUM; // Piešķir spēlētāja atribūtam myID spēlētāja numuru
    // ....
}
```

```
// Līdzīgi var veidot klases Goalkeeper konstrukturu
```

```
// Katra veida spēlētāja (piemēram, vārtsargs, aizsargs, uzbrucējs) objektu radīšanai ieteicams veidot atsevišķu klasi.
```

8. Skaidrojums par PlayerInterface.h

```
// Datu tips laukuma puses apzīmēšanai.
// Visiem vienas komandas spēlētājiem pirms spēles sākuma spēles vadības programma GameSimulator.exe
// piešķir vienu no laukuma pusēm (LEFT vai RIGHT)
enum class Side { UNDEF = 0, LEFT = 1, RIGHT = 2 };

// Laukumā esošo personu parametru datu struktūra.
// Spēlētājs var iegūt piekļuvi visu laukumā esošo personu parametriem (t. i., var "redzēt" visus pārējos spēlētājus).
// GameSimulator.exe pēc katras spēles laika vienības (ik pēc 50 milisekundēm), izmantojot metodi
// SetFieldPersonList(FieldPerson* pFP, int n) nodod katram spēlētājam rādītāju uz laukumā esošo spēlētāju
// parametru masīva sākumu (pFP) un personu skaitu (n), t. i., masīva elementu skaitu. Katrs masīva elements ir
// datu struktūra FieldPerson. Spēlētājs var izmantot šos datus savas rīcības noteikšanai (izvairīties no saskriešanās
// ar citiem, piespēlēt bumbu partnerim u. tml.).
struct FieldPerson {
    int nID;           // spēlētāja numurs
    Side side;         // spēlētāja komandas laukuma puse (kreisās vai labās puses komandas spēlētājs)
    int x, y;          // spēlētāja koordinātes laukumā
    int radius;        // spēlētāja rādiuss
};

// Abstraktā klase PlayerInterface, no kuras jāatvasina spēlētāju klase.
// Spēlētāju klasēs jāpārdefinē visas tīrās virtuālās metodes, jo tās izmantos spēles simulācijas vadības
// programma, lai sazinātos ar spēlētājiem (piemēram, nolasītu spēlētāja numuru, uzzinātu tā koordinātes utt.).
// Ik pēc vienas spēles laika vienības (50 ms) spēles simulācijas programma katram spēlētājam:
// - izpilda metodi Move(), tādējādi pieprasot spēlētājam pārrēķināt savas koordinātes,
// - izpilda metodi SetFieldPersonList(), nododot spēlētājam aktuālo informāciju par visiem abu komandu spēlētājiem,
// - izpilda metodi Draw(), lai spēlētājs uzzīmē savu attēlu.
// Pārējās spēlētāja interfeisa metodes vadības programma lieto dažādos spēles momentos, lai iegūtu
// informāciju par spēlētājiem (t. i., lai "vērotu" spēlētājus).

class PlayerInterface {
public:
    virtual int GetRadius() const = 0;           // Atgriež spēlētāja rādiusa vērtību
    virtual Side GetSide() const = 0;           // Atgriež spēlētāja laukuma pusi
    virtual int GetID() const = 0;              // Atgriež spēlētāja numuru
    virtual void SetPosition(int x, int y) = 0; // Piešķir vērtības spēlētāja koordinātēm
    virtual void GetPosition(int& x, int& y) const = 0; // Atgriež spēlētāja koordinātu vērtības
    virtual void SetFieldPersonList(FieldPerson* pFP, int n) = 0; // Nodod rādītāju piekļuvei visu
                                                                    // laukumā esošo personu parametru
                                                                    // masīvam un šo personu skaitu

    virtual void Draw() const = 0;              // Spēlētājs uzzīmē savu attēlu laukumā
    virtual void Move() = 0;                   // Spēlētājs aprēķina savas nākamās pozīcijas koordinātes
};
```

9. Skaidrojums par GameInterface.h

```
// Datu tips spēles pašreizējā statusa apzīmēšanai.
// INITIAL – spēlētāji ir sākuma pozīcijā pirms spēles sākuma
// STARTING – spēlētāji ir sākuma pozīcijā pēc vārtu guvuma
// COMPETE – pašlaik notiek komandu sacensība
// GOAL - ir iesisti vārti
// PAUSE – spēle uz laiku ir apturēta
// ENDED – spēle ir beigusies
enum class GameStatus { INITIAL, STARTING, COMPETE, GOAL, PAUSE, ENDED };

// Abstraktā klase GameInterface, kas nodrošina spēlētājiem informāciju par spēli.
// Spēles vadības programma, radot pirms spēles sākuma spēlētāja objektu ar funkcijas CreatePlayer() palīdzību, katram
// spēlētāja objektam nodod rādītāju uz spēles vadības objektu.
// Šo rādītāju pēc tam spēlētājs var izmantot, lai iegūtu informāciju par spēli, piemēram,
// uzzinātu laukuma izmērus ar metodi GetFieldSize(), uzzinātu bumbas koordinātes ar metodi GetBallCoordinates() utt.

class GameInterface {
public:
    virtual GameStatus GetStatus() const = 0; // Atgriež pašreizējo spēles stāvokli
    virtual int GetTeamSize() const = 0; // Atgriež vienas komandas spēlētāju skaitu
    virtual void GetScore(int& left, int& right) const = 0; // Atgriež pašreizējo spēles rezultātu
    virtual void GetFieldSize(int& w, int& h) const = 0; // Atgriež laukuma taisnstūra platumu un augstumu
    virtual void GetFieldCentre(int& x, int& y) const = 0; // Atgriež laukuma centra koordinātes
    virtual int GetGateWidth() const = 0; // Atgriež vārtu platumu
    virtual CDC* GetFieldDC() const = 0; // Atgriež rādītāju uz ierīces kontekstu (Device Context) laukuma taisnstūrim,
    // kas jāizmanto spēlētāja attēla zīmēšanai spēlētāja klases metodē Draw()
    virtual void GetBallCoordinates(int& x, int& y) const = 0; // Atgriež bumbas koordinātes
    virtual void KickBall(PlayerInterface* const pPlayer, int speed, double dir) = 0; // Metode, kas spēlētājam jāizmanto
    // sitenam pa bumbu, norādot parametros: rādītāju uz
    // spēlētāja objektu, kurš izdara sitienu, bumbai
    // piešķiramo ātrumu un sitiena leņķi grādos.
};
```

Papildu iespējas

1. Komandu realizācijas DLL failiem noklusētie nosaukumi ir TeamLeft.dll un TeamRight.dll, bet tos atļauts nosaukt arī citādi un izvēlēties no citas mapes uz diska. Tas ļauj ērtāk rīkot sacensības starp dažādu autoru radītām komandām. Pirms spēles (kamēr nav nospiesta poga "Open Game") var izvēlēties konkrētās DLL bibliotēkas, klikšķinot ar peles labo pogu uz komandas krāsas indikācijas laukuma.
2. Komandas vārds pēc noklusēšanas ir DLL faila vārds, un tas tiek attēlots komandas krāsas indikācijas laukumīnā. Vēlams veidot pietiekami īsus DLL failu nosaukumus, kas ietilpst šajā laukumīnā.
3. Ir iespēja piešķirt komandai citu nosaukumu, ko atgriež funkcija TeamName(), kuru varat iekļaut savā DLL. Piemērs šādai funkcijai ir atrodams parauga projektā failā Player.cpp. Ja šādas funkcijas nav, tad vadības programma par komandas vārdu izmanto DLL faila vārdu. Pievienotajā paraugā TeamLeft.dll šāda funkcija ir, un tā atgriež nosaukumu ALFA, savukārt TeamRight.dll šādas funkcijas nav un tāpēc komandas vārds ir TeamRight.
4. Pirms spēles sākuma var izvēlēties komandu krekliņu krāsas, klikšķinot ar peles labo pogu uz komandas krāsas indikācijas laukuma un izvēloties Set Color.
5. Lai spēlētāju algoritmu veidošanas laikā varētu ērtāk sekot to uzvedībai uz laukuma, ir iespējams mainīt simulācijas ātrumu, mainot standarta spēles laika vienības vērtību (50 ms) uz lielāku (ātrums smazinās) vai mazāku (ātrums palielinās). To dara, turot nospiestu Ctrl taustiņu un spiežot taustiņu - (mīnuss) vai + (pluss). Šī iespēja jāizmanto tikai spēlētāju darbības novērošanai programmas izstrādes procesā, jo nomainot spēles laika vienības vērtību, mainās arī spēles procesa uzskaitē, un tā vairs neatbilst standarta spēles gaitai.